R documentation

of all in 'man'

July 22, 2016

R topics documented:

contingencyTable 1
kappa.ct 2
kappa.set 2
rho 3
rho.ct 4
rho.minTSLength 5
rho.set 6
rhoR 7
set 8

Index **9**

`contingencyTable` *contingencyTable sample*

Description

A sample contingencyTable
ct = as.data.frame(matrix(c(3,2,1,34), nrow = 2, ncol = 2))
Displayed as:

```
|---------|
|  3 |  1 |
|----|----|
|  2 | 34 |
|---------|
```

Usage

```
contingencyTable
```

Format

An object of class `data.frame` with 2 rows and 2 columns.

Details

A contingency Table is a 2x2 matrix that contains the counts of all combinations of positive and negative ratings made by two raters.
The contingency Table is represented as a data frame: as.data.frame(matrix(c(PP, NP, PN, NN))). Displayed as:

```
|---------|
| PP | PN |
|----|----|
| NP | NN |
|---------|
```

---

kappa.ct *Calculate kappa (contingency Table)*

Description

This function calculates kappa given a contingency table.

Usage

```
## S3 method for class 'ct'
kappa(ct)
```

Arguments

ct	A contingencyTable

Value

Kappa as a double.

Examples

```
#Given a contingency Table
kappa.ct(contingencyTable)
```

---

kappa.set *Calculate kappa (Set)*

Description

This function calculates kappa given a set.

Usage

```
## S3 method for class 'set'
kappa(set)
```

Arguments

set	A set

Value

Kappa as a double.

Examples

```
# Given set
kappa.set(set)
```

---

rho *Calculate rho*

Description

This function will calculate rho given the parameters of an observed test set, which it returns as a double.

Usage

```
rho(observedKappa, codeSetBaserate, testSetLength,
  testSetBaserateInflation = 0, codeSetLength = 10000, runs = 800,
  kappaThreshold = 0.65, CSKappaMin = 0.4, precisionMin = 0.6,
  precisionMax = 1)
```

Arguments

observedKappa	The observed kappa value that will be tested with rho, as a double
codeSetBaserate	The percent, as a decimal, of the positive codes made by the first rater in the code set
testSetLength	The number of items evaluated by two raters to establish inter rater reliability
testSetBaserateInflation	The percent, as a decimal, of the test set that is guaranteed to be a positive for the first rater
codeSetLength	The length of the code sets used to calculate rho
kappaThreshold	The maximum kappa value used to generate code sets in the null hypothesis distribution for rho
CSKappaMin	The minimum kappa value used to generate code sets in the null hypothesis distribution for rho
precisionMin	The minimum precision to be used for generation of code sets in the null hypothesis distribution for rho
precisionMax	The maximum precision to be used for generation of code sets in the null hypothesis distribution for rho
replicates	The number of code sets to use in the null hypothesis distribution for rho

Value

rho, as a double

Note

replicates are similar to replicates in monte carlo.

Examples

```
# Given only required parameters
rho(observedKappa = 0.88, codeSetBaserate = 0.2, testSetLength = 80)

# Given required parameters plus testSetBaserateInflation and runs
rho(
 observedKappa = 0.88, codeSetBaserate = 0.2, testSetLength = 80,
 testSetBaserateInflation = 0.2, runs = 500
)
```

`rho.ct` *Rho (contingency Table)*

Description

This function calculates rho and kappa for a given contingency table, and returns a list containing both values.

Usage

```
rho.ct(table, codeSetBaserate = NULL, testSetBaserateInflation = 0,
  codeSetLength = 10000, runs = 800, kappaThreshold = 0.65,
  CSKappaMin = 0.4, precisionMin = 0.6, precisionMax = 1)
```

Arguments

`table`	The `contingencyTable` used to calculate rho
`codeSetBaserate`	The percent, as a decimal, of the positive codes made by the first rater in the code set, if NULL, assume the baserate of the test set is the same as the code set
`testSetBaserateInflation`	The percent, as a decimal, of the test set that is guaranteed to be a positive for the first rater
`codeSetLength`	The length of the code set used to calculate rho
`kappaThreshold`	The maximum kappa value used to generate code sets in the null hypothesis distribution for rho
`CSKappaMin`	The minimum kappa value used to generate code sets in the null hypothesis distribution for rho
`precisionMin`	The minimum precision to be used for generation of code sets in the null hypothesis distribution for rho

precisionMax	The maximum precision to be used for generation of code sets in the null hypothesis distribution for rho
replicates	The number of code sets to use in the null hypothesis distribution for rho

Value

A list containing rho and kappa

Note

replicates are similar to replicates in monte carlo.

Examples

```
# Given testSetBaserateInflation
rho.ct(contingencyTable, testSetBaserateInflation = 0.2)

# Given codeSetBaserate and testSetBaserateInflation
rho.ct(contingencyTable, codeSetBaserate = 0.1, testSetBaserateInflation = 0.2)
```

---

rho.minTSLength *Rho Min testSetLength*

Description

This function calculates the minimum testSetLength where it is possible to get a rho less than alpha, which it returns as an integer.

Usage

```
rho.minTSLength(baserate, alpha = 0.05, inc = 10, printInc = FALSE, ...)
```

Arguments

baserate	A double indicating the frequency a code occurs for the first rater in the code set
alpha	A double indicating the threshold of significance for rho (similar to an alpha level for a p value), defaulted to 0.05
inc	An integer indicating by how much the testSetLength should increase each iteration
printInc	A boolean indicating whether to print out each increment value with it's corresponding significance for rho
...	Any additional parameters passed into rho

Value

An integer giving the minimum length of testSet, as a denomination of inc, that would give a value where rho becomes mathematically possible

Examples

```
#Add testSetBaserateInflation as an additional parameter
rho.minTSLength(0.2, testSetBaserateInflation = 0.2)

#Add testSetBaserateInflation as well as changing inc and selecting printInc
rho.minTSLength(0.2, inc = 5, printInc = TRUE, testSetBaserateInflation = 0.2)
```

`rho.set` *Rho (set)*

Description

This function calculates rho and kappa for a given set, and returns a list containing both values.

Usage

```
rho.set(set, codeSetBaserate = NULL, testSetBaserateInflation = 0,
  codeSetLength = 10000, runs = 800, kappaThreshold = 0.65,
  CSKappaMin = 0.4, precisionMin = 0.6, precisionMax = 1)
```

Arguments

`set`	The test `set` used to calculate rho
`codeSetBaserate`	The percent, as a decimal, of the positive codes made by the first rater in the code set, if NULL, assume the baserate of the test set is the same as the code set
`testSetBaserateInflation`	The percent, as a decimal, of the test set that is guaranteed to be a positive for the first rater
`codeSetLength`	The length of the code set used to calculate rho
`kappaThreshold`	The maximum kappa value used to generate code sets in the null hypothesis distribution for rho
`CSKappaMin`	The minimum kappa value used to generate code sets in the null hypothesis distribution for rho
`precisionMin`	The minimum precision to be used for generation of code sets in the null hypothesis distribution for rho
`precisionMax`	The maximum precision to be used for generation of code sets in the null hypothesis distribution for rho
`replicates`	The number of code sets to use in the null hypothesis distribution for rho

Value

A list containing rho and kappa

Note

replicates are similar to replicates in monte carlo.

Examples

```
# Given testSetBaserateInflation
rho.set(set, testSetBaserateInflation = 0.2)

# Given codeSetBaserate and testSetBaserateInflation
rho.set(set, codeSetBaserate = 0.1, testSetBaserateInflation = 0.2)
```

---

rhoR *rhoR: A package for computating rho.*

Description

rho is used to test the generalization of inter rater reliability (IRR) statistics, in this case Cohen's Kappa.

Details

Rho starts with an observed kappa value, calculated on a subsample of data, known as an observed test set, and a *kappa threshold* which indicates significant agreement between raters,.

It then generates a collection of simulated, fully-coded data sets, or *code sets*, all of which have a kappa value below the kappa threshold.

Then kappa is calculated on a simulated *test set* sampled from each of the code sets in the collection to create a null hypothesis distribution. These simulated test sets mirror the observed test set in their size and sampling method.

The null hypothesis is that the observed test set, was sampled from a code set within a population of code sets which have kappas less than code set kappa max, or that the observed kappa is not representative of the level of agreement that would be reached if both raters were to code the entire data set.

Researchers can set their own alpha level. If alpha is greater than the percentage of the distribution of samples from the code sets that is greater than the observed kappa, the null hypothesis is rejected and one can conclude that two raters would have acceptable agreement had they coded the rest of the data.

rho

Use `rho.set`
`rho.ct`

kappa

Use `kappa.set`
`kappa.ct`

rho.minTSLength

Use `rho.minTSLength`

`set` *set sample*

Description

A sample test set
gold = c(1,1,1,1,rep(0,36)) silver = c(1,1,1,0,1,1,rep(0,34))
set = cbind(gold,silver)

Usage

```
set
```

Format

An object of class data.frame with 40 rows and 2 columns.

Details

A set can be any length of vectors that are bound using `cbind`. The length of each of these vectors becomes the set length

Index

∗Topic **datasets**
contingencyTable, 1
set, 8
∗Topic **kappa.ct**
kappa.ct, 2
∗Topic **kappa.set**
kappa.set, 2
∗Topic **rho.ct**
rho.ct, 4
∗Topic **rho.minTSLength**
rho.minTSLength, 5
∗Topic **rho.set**
rho.set, 6
∗Topic **rho**
rho, 3

cbind, *8*
contingencyTable, 1, *2*, *4*

kappa.ct, 2, *7*
kappa.set, 2, *7*

rho, 3, *5*
rho.ct, 4, *7*
rho.minTSLength, 5, *7*
rho.set, 6, *7*
rhoR, 7
rhoR-package *(*rhoR*)*, 7

set, *3*, *6*, 8